

libGDX

Created by <Academia de Código_>

Why LibGDX?

LibGDX

LibGDX is a powerful, **open-source** and mature game development framework that can export to multiple platforms:

Desktop | Android | Blackberry | iOS | HTML

LibGDX

It was built by the BadLogicGames studio and is the powerhouse of the Java game development frameworks

LibGDX was built on top of LWJGL, which was used to create Minecraft. That's really something!

It is vast, well **documented** and takes care of all the low-level concerns your app might have

It is also a viable mobile development framework, despite not being marketed as such

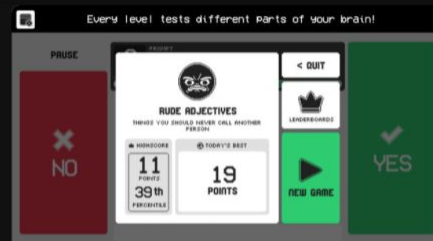
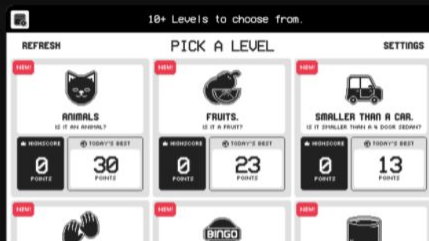
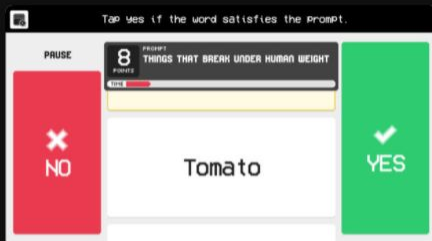
What can I do with **this**?



Crash Cards

Yes-or-no questions at high speeds

[Google Play](#) | [App Store](#)





Zombie Rush

A perfect blend of shooter and runner

Google Play



Slay the
Spire

Slay the Spire

Roguelike Card Game

Steam





Delver

Dungeon Crawler

Steam



Application Life Cycle

Right out of the gate, your core class will have 3 methods:

`create()` | `render()` | `dispose()`

You can think of `create()` as being analogous to `init()`

`render()` will act as your game loop

`dispose()` serves the purpose of removing heavy objects from our application's memory

The framework's event listener will invoke all the event methods immediately before `render()`

Android Specific

`pause()` | `resume()`

`pause()` is invoked during our application's execution when we lose focus.
e.g. during an incoming call

`resume()` is invoked when our application regains focus

Great, but how do I start?

LibGDX projects are created through the LibGDX Project Generator
which you can **download** from their website

Just run the .jar and setup your project!

Setup

Step 1 - Basic Config

Libgdx Project Generator

libGDX
PROJECT SETUP

Name: my-gdx-game

Package: com.mygdx.game

Game class: MyGdxGame

Destination: /Users/Brighenti/dev/libgdx/lecture Browse

Android SDK: /Library Browse

Sub Projects

☒ Desktop ☐ Android ☒ iOS ☐ iOS-moe ☒ HTML

Extensions

☐ Bullet ☐ Freetype ☐ Tools ☐ Controllers ☐ Box2d

☐ Box2dlights ☒ Ashley ☐ AI

Show Third Party Extensions

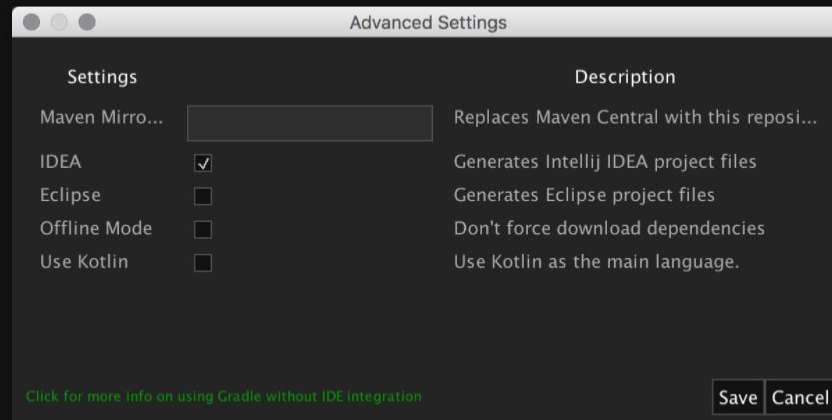
Advanced Generate

Step 1 - Basic Config

After launching the Project Generator app we must configure which modules we want to use

For now, we'll ignore the Android module

Step 2 - Advanced Config



Selecting the IDEA option will generate the IntelliJ IDEA project configuration for us

Step 3 - Opening the project on IntelliJ IDEA

BUILD SUCCESSFUL

Total time: 5.298 secs

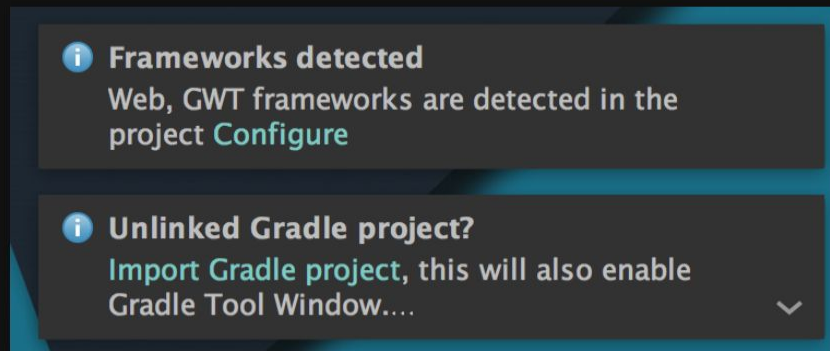
Done!

To import in Eclipse: File -> Import -> General -> Existing Projects into Workspace

To import to IntelliJ IDEA: File -> Open -> YourProject.ipr

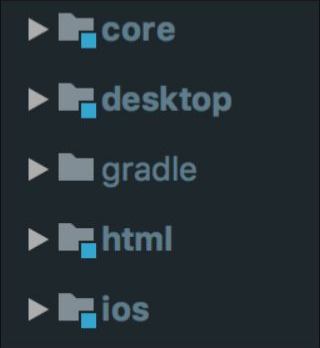
To open the project on our IDE, we'll use the
.ipr file generated in the last step

Step 4 - Gradle



LibGDX uses Gradle as a build tool. It serves the same purpose as Maven does in our projects

Step 5 - Modules

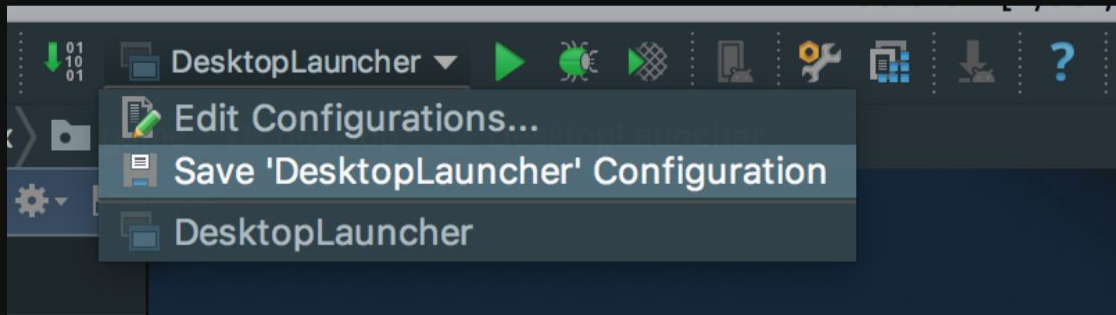


- ▶ core
- ▶ desktop
- ▶ gradle
- ▶ html
- ▶ ios

A LibGDX project is separated into various modules;
one for each platform we're targeting

Every module besides the Core module contains the module launcher.
All our application logic will be inside the Core module

Step 6.1 - Edit IntelliJ Configuration



We'll need to configure our working directory, otherwise our IDE won't be able to find our assets/resources

Step 6.2 - Configuring the working directory

Main class:	com.mygdx.game.desktop.DesktopLauncher	...
VM options:		
Program arguments:		
Working directory:	/Users/Brighenti/dev/libgdx/lecture/core/assets	... 
Environment variables:		...
Use classpath of module:	 desktop	▼
JRE:	Default (1.8 - SDK of 'desktop' module)	▼ ...

Just configure this project's working directory as the Core module assets directory and you'll be set

Android 1 - Selecting the Android Module



The screenshot shows the 'Libgdx Project Generator' window. At the top, it displays the 'libGDX PROJECT SETUP' logo. Below the logo, there are several input fields for project configuration:

- Name:** my-gdx-game
- Package:** com.mygdx.game
- Game class:** MyGdxGame
- Destination:** /Users/Brighenti/dev/libgdx/android (with a 'Browse' button)
- Android SDK:** /Users/Brighenti/Library/Android/sdk (with a 'Browse' button)

Below these fields, there are two sections:

- Sub Projects:** A row of checkboxes where 'Desktop', 'Android', 'ios', 'ios-moe', and 'Html' are all checked.
- Extensions:** A row of checkboxes where 'Bullet', 'Freetype', 'Tools', 'Controllers', 'Box2d', 'Box2dlights', 'Ashley', and 'AI' are all unchecked.

At the bottom of the form, there is a 'Show Third Party Extensions' button, followed by 'Advanced' and 'Generate' buttons.

Android 1 - Selecting the Android Module

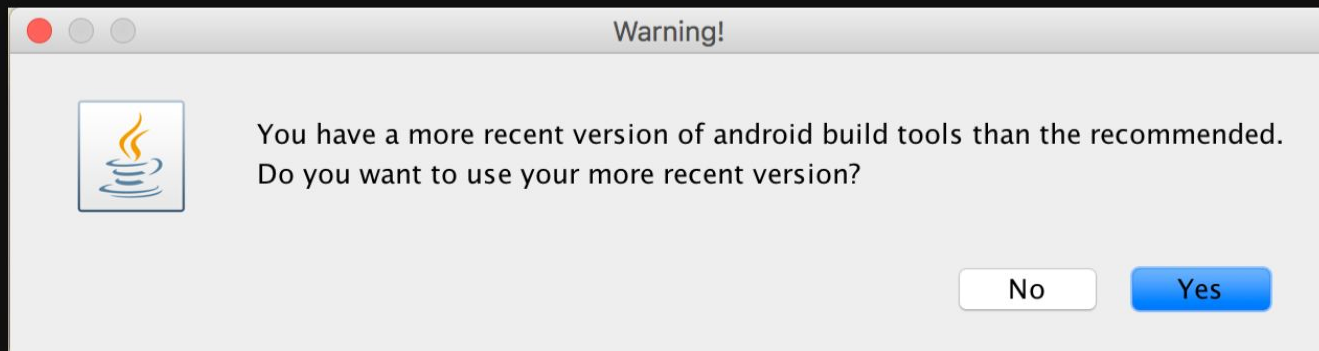
If you want your project to also target the Android platform
we'll need a couple of extra steps

Select the Android module on our Project Generator configuration

You'll need the Android SDK installed on your machine.
By default it's installed under the directory shown in the image above

Selecting the directory with Finder won't work (Library is not visible),
so you'll have to write it yourself

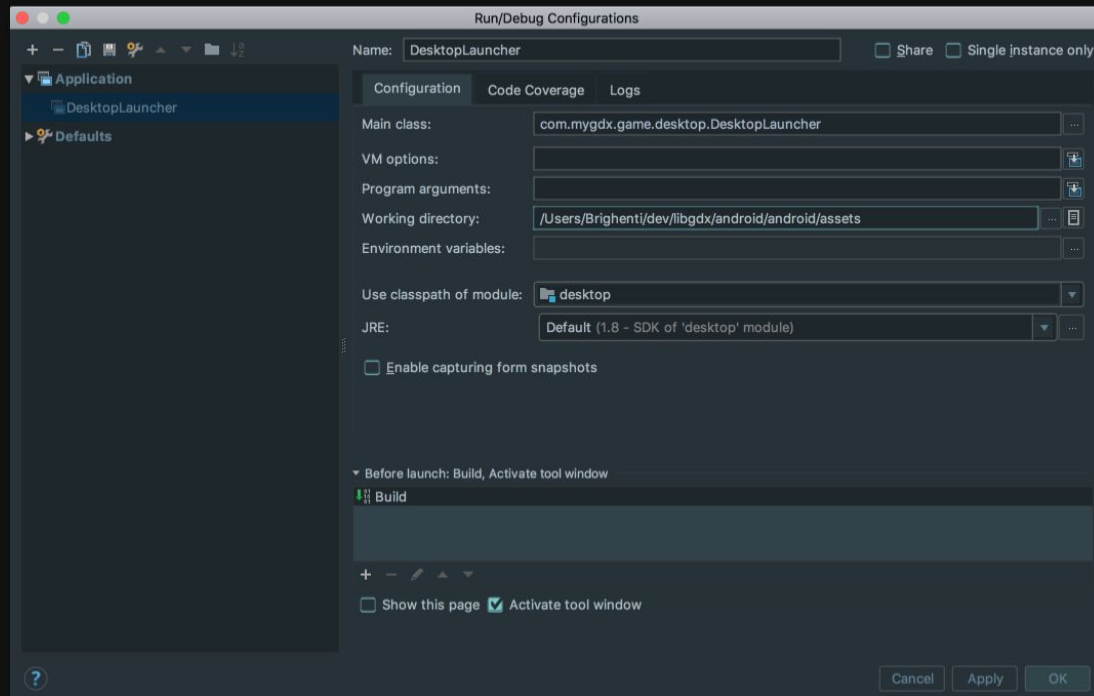
Android 2 - API Mismatch



Most likely your Android SDK is more recent than
LibGDX is expecting, so just keep saying yes!

Trust me on this one, it'll be fine

Android 3 - Working Directory



Android 3 - Working Directory

Android applications have some limitations when it comes to defining its accesses (like when you install something on your phone!)

Because of this, we need to configure our working directory as the Android module assets directory

Every other module will have access to these assets

Tutorials

Simple Game

(LibGDX Github Wiki)

Zombie Bird

(Kilobolt)

Games From Scratch

(great but slightly outdated)

Google!